

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

ANÁLISIS ARQUITECTÓNICO - Validador de Prompts y Tokens

PATRÓN ARQUITECTÓNICO: Single-File Dual-Layer (2 archivos máximo)

Dada la restricción estricta de máximo 2 archivos, se adopta una arquitectura ultra-condensada donde cada capa (frontend/backend) reside en un único artefacto autocontenido. Este patrón es válido para herramientas internas de baja escala y alta velocidad de entrega.

FRONTEND - index.html (Monolito de Presentación):

- Patrón: Single Page Application (SPA) sin framework, vanilla JS puro.
- Tailwind CSS vía CDN elimina la necesidad de archivos de estilos externos.
- Gestión de estado mínima mediante variables JS en memoria (no se requiere store).
- Flujo UX: idle !' validación local !' loading !' resultado/error.
- La validación de prompt vacío ocurre en cliente ANTES de cualquier llamada de red (fail-fast client-side).
- fetch() nativa con async/await para consumo del endpoint POST.
- Renderizado condicional del panel de resultados (oculto por defecto, visible post-respuesta).

BACKEND - main.py (FastAPI Microservicio Simulado):

- Framework: FastAPI por su soporte nativo async/await y generación automática de OpenAPI.
- Patrón: Single Endpoint Microservice, expone únicamente POST /api/analyze.
- CORS configurado explícitamente para orígenes localhost (puertos comunes: 3000, 5500, 8080) y file:// origen para apertura directa del HTML.
- Lógica de negocio: cálculo de tokens = len(words) * 1.3 (redondeado a entero).
- Simulación de latencia: asyncio.sleep(random.uniform(0.5, 2.0)) — no bloqueante.
- Estado calculado: tokens <= 1000 !' 'Óptimo', tokens > 1000 !' 'Alerta'.
- Modelo Pydantic para validación del payload entrante (campo prompt requerido, no vacío).
- Respuesta JSON estructurada: {tokens, latency_ms, status}.

SEGURIDAD (DevSecOps mínimo viable para herramienta interna):

- CORS restrictivo: solo orígenes localhost explícitos, no wildcard '*' en producción.
- Validación de input en AMBAS capas: cliente (UX) y servidor (Pydantic).
- Sin persistencia de datos: el prompt no se almacena, procesamiento stateless.
- Sin autenticación requerida (herramienta interna de red local).
- Sanitización implícita por Pydantic al tipar el campo como str con min_length=1.

TECNOLOGÍAS:

- Python 3.9+, FastAPI, Uvicorn (servidor ASGI), Pydantic v2.
- HTML5, CSS3 (Tailwind CDN), JavaScript ES2020+.
- Sin base de datos, sin caché, sin cola de mensajes.

DEPENDENCIAS PYTHON (requirements implícitos): fastapi, uvicorn[standard], python-multipart.

FLUJO DE DATOS:

Usuario escribe prompt !' validación JS (vacío?) !' POST /api/analyze con {prompt} !' FastAPI valida Pydantic !' calcula tokens !' asyncio.sleep simulado !' responde {tokens, latency_ms, status} !' frontend renderiza 3 tarjetas de resultado.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 15,991

Tokens de salida (output): 7,616

Total tokens: 23,607

Horas-hombre estimadas: ~2.6 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

Arquitecto: 1,080 input | 1,049 output | 1 llamadas

Tech Lead: 1,952 input | 3,000 output | 2 llamadas

Desarrollador: 4,904 input | 3,031 output | 2 llamadas

QA: 8,055 input | 536 output | 2 llamadas